

CHAPTER 9

STATIC MEMBERS

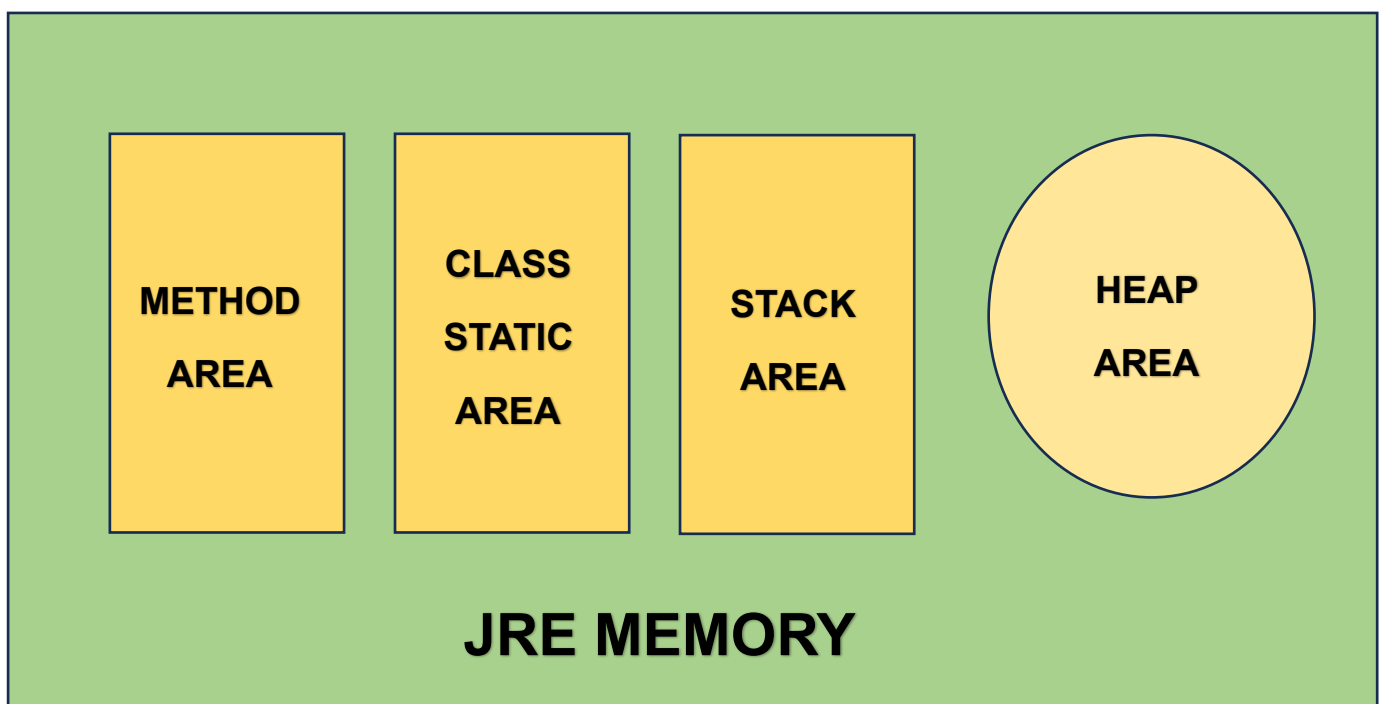
The Static members are class members declared directly in the class block. There are three types of Static members in Java as follows.

1. Static Variables
2. Static Methods
3. Static Initializers

Before understanding the static members, it is essential to get a visualization of how the internal memory is segregated by the JRE to store different class members.

Whenever the program starts with execution, the Java Runtime Environment (JRE) requests the Random Access Memory (RAM) for a memory space to execute Java programs.

The JRE segregates the memory into four areas as follows:



Method area: This area is used to store all methods and sub-blocks of the class.

Class Static area: This area is used to store the entire class and static members.

Stack area: This area is used during run time for execution memory.

Heap area: This area is used to store non-static members including objects.

CLASS LOADING PROCESS:

When a source file is compiled and is ready for execution, the generated class file has to be moved from secondary memory(ROM) to primary memory(RAM).

Inside JDK, along with JIT compiler or Interpreter, there is a component called **ClassLoader** that is responsible for the Class Loading process.

Process of Class Loading:

- The entire class is loaded to the Class Static area of the JRE segregated memory area.
- All the static members are also loaded into the Class Static area.
- The Static methods (including main method) are loaded into the method area as they are methods.
- The address/reference of those static method memory blocks along with the method signature are stored in the Class Static area.

Importance of Class Loading:

The execution of the program starts from the main method.

- ❖ Whenever the static members of the class are accessed by their name or using ClassName if accessed from other class, the control comes to the Class Static area.
- ❖ The name of the class is identified and using dot/access operator, we are able to access them.
- ❖ The control goes inside the Class memory block in the Class Static area and searches for that static variable.
- ❖ It returns the value present in the memory block of that variable.
- ❖ If not re-initialized, then default value is returned, since they are global variables.
- ❖ All these happen only because of Class Loading process.

Static members and Class Loading:

The static members of the class like,

- Static variables
- Static methods
- Static Initializers

get loaded by the Class loading process.

Thus, Static members have shallow copy/single copy of them in the internal memory. Because Class Loading happens only once for the program.

STATIC VARIABLES:

The variables declared in global scope (class block) with static keyword are called static variables. 'static' is a modifier which changes the characteristics of the variable.

SYNTAX:

```
class ClassName
{
    static datatype identifier;
}
```

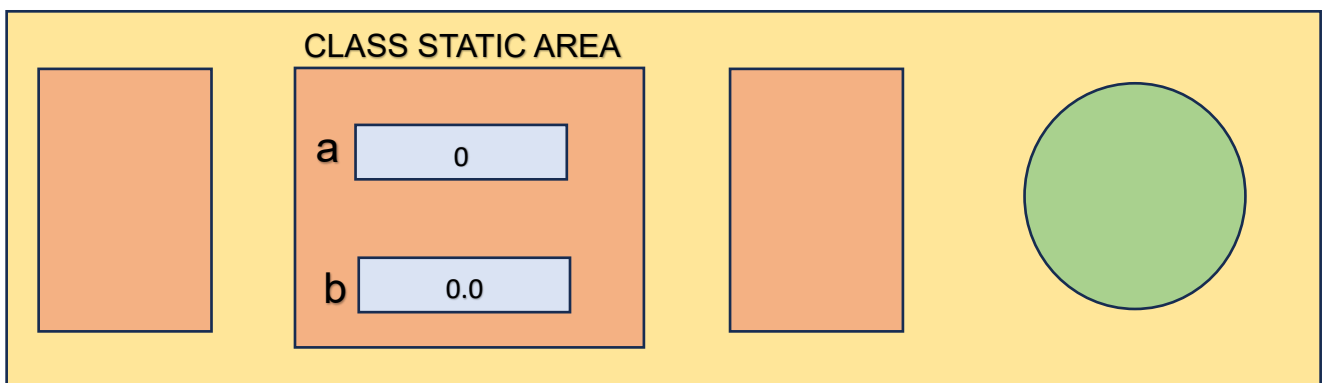
Characteristics of a Static Variable:

1. Prefixed with 'static' modifier/keyword.
2. Declared in class block and hence has global scope.
3. Stored in Class Static area.
4. Initialized with default values.
5. Accessed with name and ClassName.name
6. Any number of static variables can be declared.

Storage and Access of Static Variable:

```
1 package samplejava;
2
3 class ClassName
4 {
5     // STATIC VARIABLES
6     static int a;
7     static float b;
8
9     public static void main(String[] args)
10    {
11        System.out.println(a);
12        System.out.println(b);
13        System.out.println(ClassName.a);
14        System.out.println(ClassName.b);
15    }
16 }
17
18
```

<terminated> Cla
0
0.0
0
0.0



STATIC METHODS:

The methods which have 'static' modifier are called static methods. The most important and familiar static method is the main method.

SYNTAX:

[access modifier] static return_type method_name ([formal arguments])

```
{  
}
```

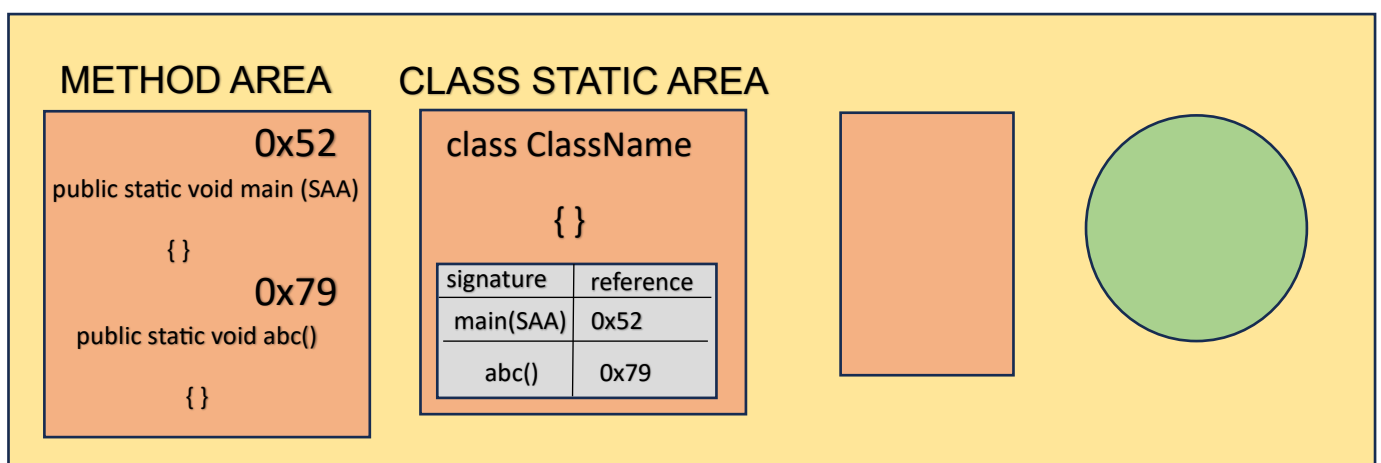
Characteristics of a Static method:

1. Must have modifier as 'static'.
2. Declared in class block like every method.
3. Storage of static method:
 - Method block along with statements in Method area
 - Signature and Reference in Class Static area
4. Accessed with method call statement:
 - method_signature;
 - ClassName.method_signature;
5. Any number of static methods can be declared.

Storage and Access of Static Method:

```
1 package samplejava;  
2  
3 class ClassName  
4 {  
5     // STATIC METHOD  
6     public static void abc()  
7     {  
8         System.out.println("static method abc");  
9     }  
10  
11     public static void main(String[] args)  
12     {  
13         abc();  
14         ClassName.abc();  
15     }  
16 }
```

<terminated> ClassName [Java Appli
static method abc
static method abc



STATIC INITIALIZERS:

Initializers in Java are used to execute start-up instructions. There are two types of Static Initializers.

- Single Line Static Initializers (SLSI)
- Multi Line Static Initializers (MLSI)

Single Line Static Initializers:

The declaration of a static variable along with initialization is called as SLSI.

SYNTAX:

static datatype identifier = value;

Characteristics of SLSI:

1. They have 'static' modifier/keyword and must be initialized with a value.
2. They are declared in the class block and hence global scope.
3. They are stored in the Class Static area with the value initialized.
4. They can be accessed with name and ClassName.name
5. Any number of SLSI can be declared.

Multi Line Static Initializers:

'static' un-named block to code multiple statements of start-up instructions.

SYNTAX:

```
static  
{  
    //statements;  
}
```

Characteristics of MLSI:

1. They have 'static' modifier/keyword, along with an un-named block.
2. They are declared in the class block and hence global scope.
3. Storage of MLSI:
 - The block along with statements in Method area
 - Reference in Class Static area (since no name)
4. Accessing MLSI: They get executed implicitly and cannot be called
5. Any number of MLSI can be declared

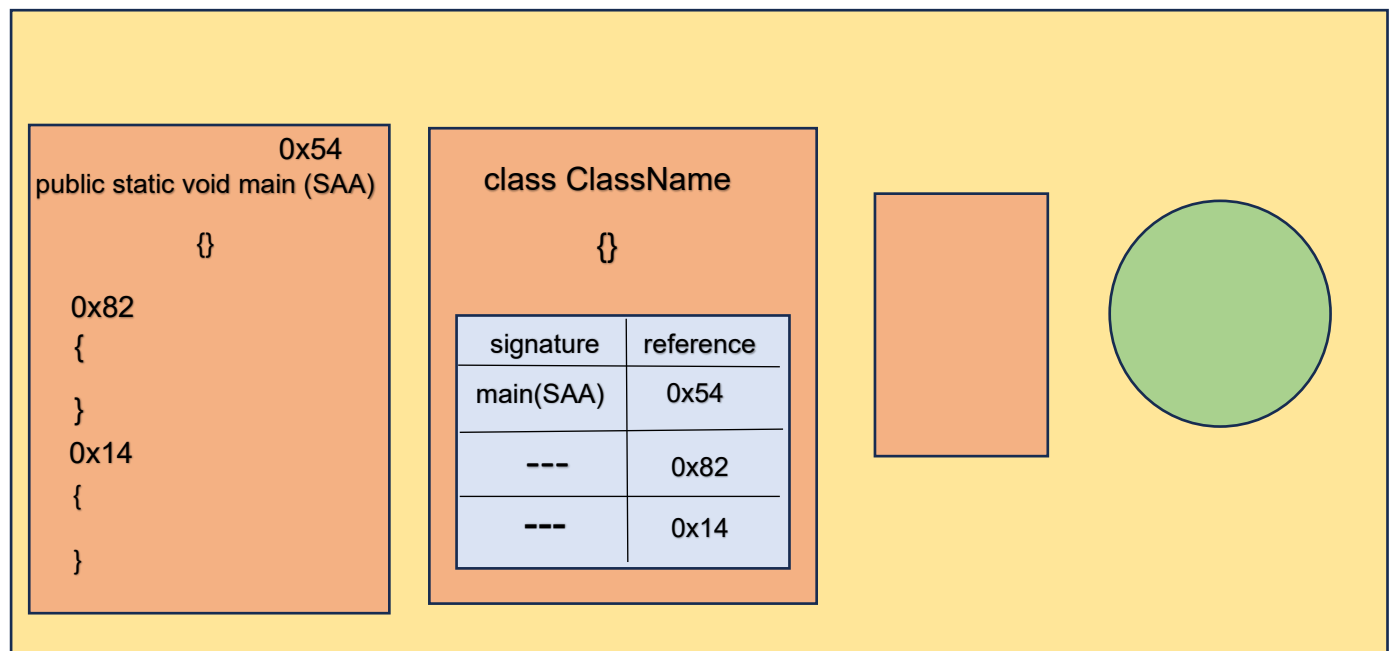
Speciality of MLSI:

- They get executed even before the main method.
- MLSI get implicitly executed only once as they are static members.
- If there are more than one MLSI, they get executed in top-down approach.

SLSI and MLSI example:

```
1 package samplejava;
2
3 class ClassName
4 {
5     // STATIC INITIALIZERS
6     static int a = 12; //SLSI
7     static String s = "abc"; //SLSI
8     static //MLSI
9     {
10         System.out.println("MLSI one");
11     }
12     public static void main(String[] args)
13     {
14         System.out.println(a);
15         System.out.println(s);
16         System.out.println(ClassName.a);
17         System.out.println(ClassName.s);
18     }
19     static //MLSI
20     {
21         System.out.println("MLSI two");
22     }
23 }
24
```

```
<terminated> ClassName [
MLSI one
MLSI two
12
abc
12
abc
```



These are the three static members in Java.